計算物理学II

第4回
数値計算プログラミング入門（1）：
変数, 型, 四則演算

秋山 進一郎

2026年10月23日

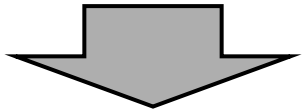
授業日の確認

- 全10回
  - 第1回：10月2日（金）
  - 第2回：10月9日（金）
  - 第3回：10月16日（金）
  - **第4回：10月23日（金）**
  - 第5回：11月6日（金）
  - 第6回：11月13日（金）
  - 第7回：11月20日（金）
  - 第8回：12月4日（金）
  - 第9回：12月11日（金）
  - 第10回：12月18日（金）

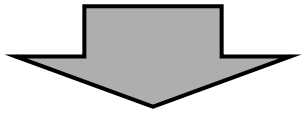
本日10月23日は自習とします．各自で演習課題に取り組んでください

毎回の授業の流れ

“compphy2”に移動する



“git pull origin main”で演習資料を入手



“lecture4”の中の資料を開いて演習開始

本日の演習内容

- Pythonを通して, 数値プログラミングの基礎的事項を習得する
  - 変数, 型について理解する
  - プログラミングの記法と数学的記法に違いがあることを理解する
  - Pythonを使って, 四則演算を行う

この授業では「Python3」を使います

- Pythonには2系（Python2）と3系（Python3）がある
  - Python2のサポートは2020年に切れている
  - 特別な理由がない限りはPython3を使う

Python3が使えることを確認

- Terminalから以下のコマンドを実行しよう
  - Python3のインストールされている場所（パス）を確認できます

```
akiyama@~/compphys2/lecture4$ which python3
/opt/homebrew/bin/python3
```

  - 全学計算機では上とは違うパスが返ってきます

- 今度は以下のコマンドを実行し，Python3のバージョンを確認しましょう

```
akiyama@~/compphys2/lecture4$ python3 --version
Python 3.14.7
```

使えるライブラリの確認

・今度は以下のコマンドを実行しよう

```
akiyama@~/compphys2/lecture4$ pip freeze
certifi==2026.7.22
contourpy==1.3.3
cycler==0.12.1
fonttools==4.61.0
kiwisolver==1.4.9
matplotlib==3.10.8
numpy==2.5.3
packaging==25.0
pillow==12.1.0
```

・全学計算機に用意されているPythonのライブラリのリストを見ることができます

・例えば，matplotlib, numpy, scipyなどのライブラリが入っているか確認しましょう

・ライブラリはアルファベット順にリストされていることに注意

全学計算機でのライブラリのインストール方法

- すでに第2回演習で経験済みの人もいますが, 全学計算機では以下のようにしてPythonのライブラリをインストールできます
  - 例えば, 先ほどの pip freeze の結果, numpyがインストールされていなかった人は, 以下のようにしてインストールできます

```
akiyama@~/compphys2/lecture4$ pip install --user numpy
```

  - 後半の授業では, numpyとscipyを使う予定なので, この二つが入っていない人は今のうちにインストールしておきましょう

対話モードを使ってみる

- TerminalにPython3と打ってEnterを押すと対話モードが立ち上がります
  - 対話モードを立ち上げてみましょう
  - 以下のように“>>>”が現れればOKです

```
akiyama@~/compphys2/lecture4$ python3
Python 3.14.7 (main, Aug  5 2026, 10:29:49) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

変数とは？

- 任意のプログラミング言語において，「変数」という概念が現れる
- そもそも計算機とは，メモリ上にあるデータをCPUで処理し，その結果を再度メモリ上に戻す作業をする機械である
  - このメモリには「番地」のような通し番号が付いている
  - 例：計算機上での足し算は，「0番地と1番地のデータを読み込み，それらを足し算し，その結果を2番地に書き込め」ということをやっている
- 人間にとって，上のようにメモリの「番地」を使ってプログラムを書くのはとても面倒
- そこで「番地」の代わりに使うのが「変数」と呼ばれるラベル付けされた“箱”

型とは？

- 任意のプログラミング言語において，「型」という概念が現れる
- そもそも計算機上のメモリには整数しか保存できない
- では, 小数点を含む数, 真偽値, 文字列などはどうやって扱っているのか？

  ⇒「メモリの値をどう解釈するか」というある種のルールを指定することで, 整数値以外を表現している

  - この「メモリの値をどう解釈するか」というルールのことを「型」と呼ぶ

- 全ての「変数」は何らかの「型」を持っている

具体的な型について

- 型とは, データの種類のことと思えば良い
  - 型の例：整数(int)型, 浮動小数点(float)型, 複素数(complex)型, 文字列(str)型
  - int型, complex型, str型はその名前の通り, 整数, 複素数, 文字列のことです
  - float型は聞きなれないと思いますが, 「小数点を含む数」のことで, 数値計算では極めて重要な型の一つです

変数に値を代入する

・aという変数を用意し，aに1というint型の値を代入してみよう

  ・「代入」は「=」によって行う

  ・a=1というプログラムを書き, Enterを押して実行する
  （実行しても特に出力はありません）

```
>>> a=1
```

・このプログラムは, “a”という変数を作り, そこに1というint型を代入する, という意味になります

・計算機の中では, メモリ上に“a”というラベルを貼り, そこに1を書き込む, という処理が行われています

print()とtype()

・printという命令を使うと, 変数が指している値（変数の中身）を確認できます

　・print(a)と書いて実行してみよう

・typeという命令を使うと, 変数の型を確認できます

　・type(a)と書いて実行してみよう

　・<class ‘int’>は変数aがint型である, という意味です

```
>>> print(a)
1
>>> type(a)
<class 'int'>
```

変数の定義と中身の更新 1/2

- 新たに, bという変数を用意し, 2を代入するプログラムを書いて実行しよう

```
>>> b=2
```

- 次に, c=a+bというプログラムを書き, 実行しよう

```
>>> c=a+b
```

  - このプログラムでは, まず右にあるa+bという足し算(+)が実行され, 新たな変数cを用意し, a+bの結果として得られる数値(今の場合は3)をcに書き込む, という処理が行われる

- c=a+bというプログラムでは, cは未定義ですが, Pythonでは代入(=)が行われる際に, =の左側にある変数が未定義であれば新規作成され, 定義済みならその変数の中身を更新します

変数の定義と中身の更新 2/2

- printを使って, 先ほどの変数cの中身を確認します

```
>>> print(c)
3
```

- 今度は, c=c+aというプログラムを書き, 実行し, printを使って変数cの中身を見てみましょう

```
>>> c=c+a
>>> print(c)
4
```

  - このプログラムでは, まず右にあるc+aという足し算(+)が実行され, 定義済みの変数cの中身を, c+aの結果として得られる数値(今の場合は4)によって更新する, という処理が行われる

- つまり, プログラミングにおける代入「=」は, 数学の等号記号とは全く意味が違います

未定義変数は「=」の右側に置かない

- Pythonでは代入(=)が行われる際に, =の左側にある変数が未定義であれば新規作成され, 定義済みならその変数の中身を更新します

- 一方, =の右側に未定義の変数を置くことはできません

- c=dというプログラムを書き, 実行してみましょう

```
Traceback (most recent call last):
  File "<python-input-14>", line 1, in <module>
    c=d
      ^
NameError: name 'd' is not defined. Did you mean: 'id'?
```

- “name ‘d’ is not defined”というエラーになります

  - c=dというプログラムを実行した時点では, dが指す番地が指定されていないから

未定義変数は「=」の左側に置く

- 先ほどのc=dというプログラムはエラーになりましたが, d=cというプログラムは動作します

```
>>> d=c
>>> print(d)
4
```

- このプログラムでは, 未定義のdが=の左側にあり, 定義済みのcが=の右側にあるため, エラーは生じません
  - この例を通しても, プログラミングにおける代入「=」は, 数学の等号記号とは全く意味が違いますことが明らかでしょう

異なる型同士の演算 1/2

- 異なる型同士を演算した場合, 結果がどのような型で与えられるのかに注意が必要です
- ここでは, 変数を使わずに数値を直接使っていきます
- 以下のプログラムを実行してみましょう
  - 「1」はint型, 「1.0」は小数点を含むのでfloat型になっています

```
>>> type(1)
<class 'int'>
>>> type(1.0)
<class 'float'>
```

異なる型同士の演算 2/2

- typeの引数には, 演算式を渡すこともできます
- 以下のプログラムを実行しましょう

```
>>> type(1+1)
<class 'int'>
```

```
>>> type(1.0+1.0)
<class 'float'>
```

- 同じ型同士を演算した結果も, 同じ型を持つことが分かります
- 続いて, 以下のプログラムを実行しましょう

```
>>> type(1+1.0)
```

- int型とfloat型を足すと, その結果はfloat型で与えられます

掛け算の場合

- 今度は, 足し算ではなく, 掛け算をやってみましょう
  - Pythonでの掛け算は「＊」（アスタリスク）を使います
- 足し算の時と同様, typeを使って, 以下の三つを具体的な計算で確認してみてください
  - int ＊ int ＝ int
  - float ＊ float ＝ float
  - int ＊ float ＝ float

割り算について

- Pythonでの割り算は「/」を使います
  - int型, float型の範囲で「/」 を使うと, 割り算の結果は必ずfloat型で返されます
- 左下の三つを実行し, 全ての結果が小数点を含む数になっていることを確認しましょう
- 念のため, 右下の三つも実行し, typeが返してくる型を確認しましょう

```
>>> 4/2
2.0
>>> 4.0/2
2.0
>>> 4.0/2.0
2.0
```

```
>>> type(4/2)
<class 'float'>
>>> type(4.0/2)
<class 'float'>
>>> type(4.0/2.0)
<class 'float'>
```

割り算に関する注

- Pythonでは「//」を使うと，割り算の結果の小数部分を切り捨てた値が得られます
  - 「/」とは異なり，「//」による結果は必ずしもfloat型とは限りません
- 左下の三つを実行し，このことを確認しておきましょう

```
>>> 5//2
2
>>> 5.0//2
2.0
>>> 5.0//2.0
2.0
```

浮動小数点数を扱う時の注意 1/2

- そもそも, 浮動小数点数とは, 実数を有限桁の小数として近似表現したものである
  - そのため, 浮動小数点数には誤差がある
- 浮動小数点数には誤差があることが分かる例を挙げます
  - まず, 次のようなfloat型同士の足し算をしましょう

```
>>> 0.1+0.1
0.2
```

  - 何の問題もありませんね. それでは次のfloat型同士の足し算は？

```
>>> 0.2+0.1
```

浮動小数点数を扱う時の注意 2/2

- 0.2+0.1は0.3, にはならず, 下のように僅かにズレることが分かったでしょうか

```
0.30000000000000004
```

- 実はこの結果は, 計算機が二進数表現に基づいていることと,浮動小数点数が実数を有限桁の小数として近似した数であることから理解できます
  - 0.1を二進数で表現すると？（時間のある時に考えてみましょう）
  - この点については, 第6回の演習で再訪します
- 数値計算では, 有限桁の計算しか行えないことをよく覚えておいてください

bool型

- boolとは真偽値のことで, 真(True)と偽(False)の二値をとる型のことです
- プログラミングでは, 何かと何かを比較する処理を非常によく使います
  - 比較をした結果は, 真偽値がbool型で返されます
  - プログラミングで,「何かと何かを比較する」ためには「比較演算子」を使います
  - 代表的な比較演算子として,「==」は二つの値が等しいかどうかを判定します

比較演算を使ってみる

・右のように，「==」による比較演算をやってみましょう

```
>>> 1==1
True
>>> 1==2
False
```

- 「1と1は等しい」という命題は真なので，「1==1」の結果はTrue
- 「1と2は等しい」という命題は偽なので，「1==2」の結果はFalse

・typeを使い，これらの比較演算結果がbool型を持つことも確認しておきましょう

```
>>> type(1==1)
<class 'bool'>
>>> type(1==2)
<class 'bool'>
```

float型同士の等号比較には要注意！

- bool型は, 今後学ぶ「条件分岐」や「ループ」といった処理を行うのに必須ですが, 今日の段階で一つだけ肝に銘じておいて欲しいことがあります. それは, …
  - float型には誤差があるため, 一般にfloat型同士の等号比較演算の結果は信頼してはいけません
- 下の具体例を実行しましょう

```
>>> 0.3==0.2+0.1
False
```

- 「0.3と0.2+0.1は等しい」という命題は数学的には真
- しかし, 前に見たように, float型の計算では「0.2+0.1=0.3000…0004」であり, 「0.3と0.3000…0004は等しい」という命題は偽なので, 「0.3==0.2+0.1」はFalse

complex型

- Pythonでは複素数も取り扱うことができます
  - 複素数は, complex型によって表現されます
  - Pythonでは, 数値の末尾に「j」を付けると, その数値は虚部として扱われます

- real, imagで複素数の実部，虚部を取り出せる
  - (2+1j).realと入力してみよう
  - (2+1j).imagと入力してみよう
  - 複素数の実部，虚部は整数値で入力しても浮動小数点数になることに注意しよう

complex型

- Pythonでは複素数も取り扱うことができます
  - 複素数は, complex型によって表現されます
  - Pythonでは, 数値の末尾に「j」を付けると, その数値は虚部として扱われます
- 具体例として, 実部が2, 虚部が1の複素数は「2+1j」のように表現します
  - 数値の末尾に「j」を付ける必要があるため, 「2+j」のように表現することはできません
  - 「2+j」のように書いてしまうと, この「j」は変数と見做されます
- 以下の三つを実行し, これらのことを確認しておきましょう

```
>>> type(2+1j)
<class 'complex'>
```

```
>>> 2+1j
(2+1j)
```

```
>>> 2+j
Traceback (most recent call last):
  File "<python-input-60>", line 1, in <module>
    2+j
      ^
NameError: name 'j' is not defined
```

complex型を含んだ四則演算

・complex型についても, 四則演算を行うことができます

　・complex型を含んだ四則演算の結果は, 基本的にcomplex型になります

・以下のいくつかのcomplex型を含んだ計算をやってみましょう

　・それぞれについて, typeを使い, その結果の型も確認してみてください

```
>>> (2+1j)+(2-1j)
(4+0j)
>>> (2+1j)*(2-1j)
(5+0j)
>>> (2+1j)/(2-1j)
(0.6+0.8j)
```

```
>>> (2+1j)/2
(1+0.5j)
>>> 0.3*(2+1j)
(0.6+0.3j)
>>> (2+1j)-5//2
1j
```

complex型から実部, 虚部を抜き出す

- complex型の数値に「.real」を付けることで, 実部を得ることができる
  - 同様に, 「.imag」を付けると, 虚部を得ることができる
- 実際に, 以下のように実部と虚部を抜き出しましょう

```
>>> (2+1j).real
2.0
>>> (2+1j).imag
1.0
```

- complex型では, 整数でその実部, 虚部を書いても, それらはfloat型を持ちます

str型

- Pythonでは, “ ” (ダブルクォーテーション)や ‘ ’ (クォーテーション)で囲まれた文字は文字列(str)型として扱われます

- 変数aに, str型の文字列“1”を代入してみましょう

```
>>> a="1"
```

- printを使って, aの中身を見てみます

```
>>> print(a)
1
```

- この出力結果だけを見ると, 1という整数が変数aに代入されているように見えます

  - typeを使って, aの型を確認しましょう

```
>>> type(a)
<class 'str'>
```

  - aの型は確かにstr型になっています

  - aに代入されている「1」は整数ではなく, 文字になっているということです

str型同士の足し算

・str型同士は足し算をすることができます

・以下のようなstr型同士の足し算を実行してみよう

```
>>> a="Hello "
>>> b="World!"
>>> print(a+b)
Hello World!
```

・なお, str型と数値の間で演算をすることはできません

型の変換

- ある型を別の型へと変換できるケースがあります
- よく使う例として,「int」で囲むとint型へ,「float」で囲むとfloat型へ変換できます
- 以下の例を実行してみましょう

```
>>> a=1.0
>>> b=int(a)
>>> print(b)
1
>>> type(b)
<class 'int'>
```

```
>>> a=10
>>> b=float(a)
>>> print(b)
10.0
>>> type(b)
<class 'float'>
```

```
>>> a="1"
>>> b=int(a)+2
>>> print(b)
3
>>> type(b)
<class 'int'>
```

対話モードの終了

- Pythonの対話モードは, exit()で終了できます
  - 対話モードを終了しましょう

```
>>> exit()
akiyama@~/compphys2/lecture4$
```

ここまで終わったら

- お疲れ様でした
- 今後の授業では, Pythonによる数値計算プログラミングに関する演習を行います
  - 予復習の際の副読本として, 簡単なドキュメントをお配りしています

```
akiyama@~/compphys2/python/quick_ref_python$ ls
fig                    latexmkrc              main.tex               quick_ref_python.pdf
```

- 前回の授業でVS Codeを使ったLaTeXの設定ができなかった人は, lecture_material_3.pdfの演習に再度取り組んでみてください
  - (manabaで配布済ですが) lecture_material_3.pdfのp.35に, VS CodeでLaTeXのコンパイルができない場合の対処法を掲載しましたので, そこから前回の演習をリスタートしてみてください

第1回レポート課題について

- lecture4の中に, 第1回レポート課題が入っています
- LaTeXの設定が完了した人は, 第1回レポート課題の内容を確認し, 残りの時間を使ってレポートに取り組んでください
  - レポートの提出期限は当初予定から1週間延長し, 11/16としました
  - LaTeXの設定がまだ終わらない人は, 11/6の第5回授業などで対応します

```
akiyama@~/compphys2/lecture4$ ls
lecture_material_4.pdf report1                    report1.pdf
```

レポート問題のソースファイル

レポート問題